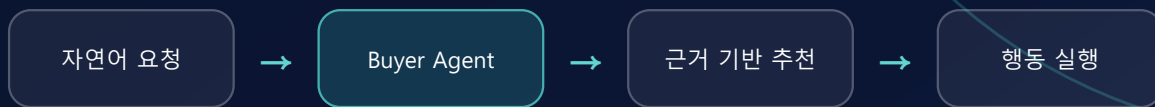


Jarvis AI 최종 보고서

자연어 상품 탐색부터 개인화 추천과 장바구니 실행까지
연결하는 에이전틱 커머스 AI 서버



요약

Jarvis는 검색어를 입력하고 결과를 고르는 기존 쇼핑 흐름을, 사용자의 상황과 조건을 이해하고 후속 행동까지 수행하는 대화형 흐름으로 확장한 AI 에이전트 서버다. 구매자 Agent는 추천·장바구니·찜·주문 조회를, 판매자 Agent는 통계 조회·분석·상품 변경 draft를 담당한다. 개인화 파이프라인은 대화와 구매 이력에서 장기적으로 유효한 취향만 선별해 다음 추천에 활용한다.

핵심 설계 원칙

LLM이 모든 결정을 하게 두지 않는다. LLM은 의미를 해석하고, 코드는 권한·제약·후보 ID·근거를 검증한다.

450

구매자 adversarial minimal-mutation case

0 / 420

validated confirmation의 등록된 unsupported evidence

10 / 10

판매자 trigger 이상 주입 회귀 케이스 통과

프로젝트가 해결한 것

탐색의 연속성

상황 중심 발화를 구조화 조건과 semantic query로 바꾸고, 멀티턴 조건을 스레드 상태로 유지한다.

행동의 연결

“두 번째 상품 담아줘”처럼 화면과 이전 추천을 참조하는 발화를 실제 Spring 트랜잭션으로 연결한다.

설명의 신뢰성

추천 후보 밖 ID와 근거 없는 수치·속성을 결정론적 validator와 평가 하네스로 차단한다.

구현 범위와 경계

이 저장소는 React 프론트엔드와 Spring 커머스 백엔드 사이의 **AI 판단·조율 계층**을 구현한다. 상품·주문·장바구니 원본과 트랜잭션 권위는 Spring에 두고, AI 서버는 대화 상태, 검색용 AI 산출물, 프로필, 추천 목록을 관리한다. 따라서 본 보고서의 성과는 AI 서버의 구현·검증 결과이며 전체 제품의 실제 매출 효과나 사용자 만족도를 의미하지 않는다.

근거: [README.md](#), [evals/adversarial_recommendation/README.md](#), [evals/rerank_grounding/README.md](#), [evals/seller_trigger/](#)

목차

문제에서 설계로, 설계에서 구현과 검증으로 이어지는 순서로 구성했다.

01	프로젝트 개요	소개 · 배경 · 목적 · 역할 경계
02	문제 정의 및 요구사항	기존 경험의 한계 · 해결 목표 · 품질 속성
03	서비스·시스템 설계	서비스 흐름 · 3-tier 아키텍처 · 기술 스택
04	AI Agent 설계	Buyer · Seller · Profile · LLM tier
05	핵심 기능 구현	추천 · 장바구니 · 개인화 · 판매자
06	데이터터 및 검색 시스템	Embedding · pgvector · reranking · 데이터셋
07	테스트 및 결과	회귀 · adversarial · live 평가 · 성능
08	결론 및 향후 개선	성과 · 한계 · 발전 가능성

읽는 방법

파란색 표와 카드는 구현 구조를, 청록색 강조는 설계 원칙을, 붉은색 한계 상자는 평가 범위를 나타낸다. 수치는 모두 저장소에 보존된 artifact를 기준으로 하며, 각 장 끝에 대표 근거 경로를 표기했다.

핵심 기술 키워드

Python 3.12 FastAPI LangGraph SSE PostgreSQL pgvector Embedding Pydantic v2 pytest Ruff

1

1. 프로젝트 개요

검색 결과를 나열하는 서버가 아니라, 사용자의 의도를 이해하고 안전하게 행동까지 이어 주는 AI 조율 계층을 구축했다.

1.1 프로젝트 소개

Jarvis는 자연어 대화로 상품을 찾고, 조건을 좁히고, 개인 취향에 맞게 순서를 조정한 뒤, 장바구니·찜·주문 조회 같은 후속 행동까지 연결하는 에이전틱 커머스 AI다. 일반적인 상품 검색이 “검정 파우치”처럼 짧은 키워드에 최적화되어 있다면 Jarvis는 “유럽 여행에 서 기내에 들고 갈 파우치와 어댑터를 총 8만 원 아래로 추천해 줘” 같은 상황·복수 니즈·예산이 섞인 요청을 다룬다.

시스템의 목표는 LLM을 쇼핑 데이터의 새로운 권위자로 만드는 것이 아니다. LLM은 문장의 의미와 선호 신호를 해석하고, 최신 가격·재고·주문 상태와 실제 변경 권한은 기존 커머스 시스템이 유지한다. 이 원칙은 추천 품질뿐 아니라 보안, 장애 격리, 팀 간 협업을 동시에 만족시키기 위한 출발점이다.

1.2 추진 배경과 목적

기존 경험

- 상황을 여러 키워드로 다시 번역해야 한다.
- 이전 조건을 검색할 때마다 반복한다.
- 추천 이유와 실제 상품 데이터가 분리될 수 있다.
- 추천 뒤 행동을 화면에서 다시 수행한다.

Jarvis의 목표

- 상황을 조건·니즈·semantic query로 구조화한다.
- 멀티턴 상태와 장기 프로필을 분리한다.
- 후보·제약·근거를 코드로 재검증한다.
- 대화를 실제 커머스 Tool 호출로 연결한다.

1.3 시스템 책임 분담

3-tier 구성요소별 책임

구성요소	핵심 책임	권위 데이터
React FE	채팅 UI, SSE 소비, 조건·상품 카드 표현	사용자에게 보이는 현재 화면 상태
Jarvis AI	의도 라우팅, 추천·개인화, Tool 조율, SSE 생성	대화 상태, AI 생성물, 프로필, 추천 목록
Spring BE	회원·상품·주문·장바구니·찜 트랜잭션	가격, 재고, 상품 원본, 커머스 거래

프로젝트의 한 문장 정의

Jarvis는 커머스 원본을 대체하지 않고, 자연어 의도를 신뢰 가능한 조회와 행동으로 번역하는 AI Agent 서버다.

근거: [README.md](#) 프로젝트 개요·시스템 아키텍처, [app/agents/](#), [app/api/](#)

2. 문제 정의 및 요구사항

자연어의 유연함을 살리면서도 상품·권한·수치의 경계는 결정론적으로 지켜야 한다.

2.1 해결해야 할 다섯 가지 문제

문제	실패 예시	필요한 대응
상황 중심 탐색	“여행 준비물”이 하나의 카테고리 축소됨	복수 니즈 분해와 category fan-out
멀티턴 조건	이전 가격·브랜드 조건을 잊거나 과도하게 승계	스레드 상태, 현재 발화 우선, 조건 제거
개인화 충돌	과거 취향이 현재 명시 요청을 덮음	현재 조건 우선, 장기 프로필은 보조 신호
근거 환각	후보 데이터에 없는 평점·리뷰·가격을 설명	구조화 근거와 deterministic validator
행동 대상 오류	화면 밖 상품 또는 잘못된 순번을 장바구니에 추가	허용 ID 집합, 화면 참조 해소, 옵션 되묻기

2.2 기능 요구사항

구매자 기능

1. 자연어를 카테고리·가격·브랜드·색상·속성과 semantic query로 분해한다.
2. 정확 조건은 검색·사후필터에서 강제하고 의미 적합성은 rerank한다.
3. 추천 결과와 화면 상태를 이용해 장바구니·찜 대상과 옵션을 확인한다.
4. 주문 상태와 일반 질의를 별도 lane으로 안전하게 분리한다.

판매자·프로필 기능

1. 판매자 범위 안의 통계와 저장된 보고서를 조회한다.
2. 상품 변경은 바로 실행하지 않고 검토 가능한 draft로 제안한다.
3. 반복성·현재성·명시성 gate를 통과한 취향만 장기 프로필에 승격한다.
4. 명시적 session-end와 비활동 종료를 공통 finalizer로 처리한다.

2.3 비기능 요구사항

Security

JWT sub에서만 신원을 도출하고 seller scope와 내부 토큰을 검증한다.

Reliability

LLM·검색·push 실패를 명시적 error 또는 degrade로 전환하고 취소를 전파한다.

Reproducibility

고정 fixture, dataset hash, prompt hash, run manifest로 실험 조건을 보존한다.

Streaming

SSE로 진행 상태와 token을 보내며 thread별 활성 스트림 수명을 제어한다.

Contract

Pydantic camelCase 모델과 API 명세로 FE·AI·BE 와이어 형식을 고정한다.

Observability

lane, provider, latency, success/degrade를 구조화 trace와 artifact에 남긴다.

성공 기준의 분리

“응답이 생성됨”, “규칙 위반이 없음”, “추천 순위가 좋음”, “사용자가 만족함”은 서로 다른 기준이다. 본 프로젝트는 회귀·제약·근거·지연을 각각 측정하며, 실제 사용자 만족도는 향후 검증 과제로 남긴다.

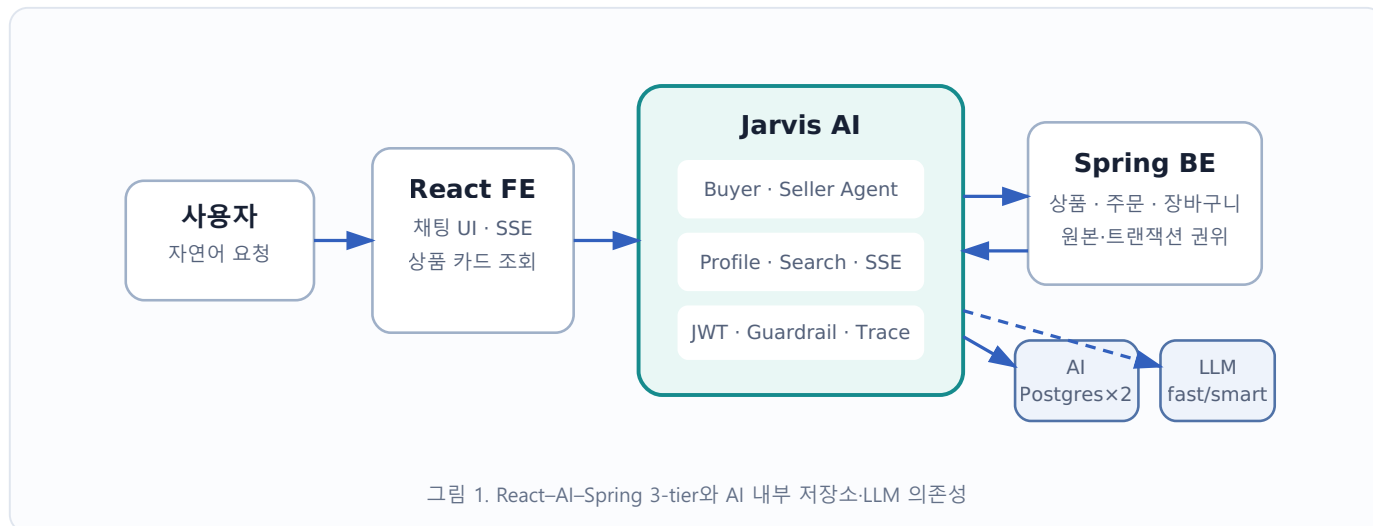
근거: [docs/api-spec.md](#), [evals/README.md](#), [app/core/auth.py](#), [app/core/tracing.py](#)

3

3. 서비스·시스템 설계

데이터 권위와 AI 판단을 분리하고, 대화·검색·트랜잭션을 명시적인 신뢰 경계로 연결했다.

3.1 전체 아키텍처



3.2 요청에서 화면까지의 서비스 흐름



FE는 Spring에서 발급받은 토큰으로 AI 서버를 직접 호출한다. AI는 요청 본문의 사용자 ID를 신뢰하지 않고 검증된 [sub](#)에서 신원을 도출한다. 추천이 끝나면 AI는 순위 목록을 Spring에 push하고 [products.ready](#)를 보낸다. FE는 이 신호를 받아 Spring에서 최신 가격·재고·이미지를 조회한다. 이 “경로 B”는 SSE 페이로드 중복을 줄이고 표시 데이터의 권위를 한곳에 둔다.

데이터 소유 원칙

Spring은 커머스 원본을, AI PostgreSQL은 search document·embedding·profile·checkpoint 같은 AI 산출물을 소유한다.

3.3 기술 스택과 선택 이유

영역	기술	선택 이유
Runtime	Python 3.12 · uv	LLM 생태계, async, 타입 힌트와 lockfile 재현성
API	FastAPI · Uvicorn · SSE	비동기 Tool 호출과 토큰·진행 상태 스트리밍
Agent	LangGraph	조건 분기, 멀티턴 상태, 서브그래프 경계를 코드로 명시
Schema	Pydantic v2	camelCase 와이어 계약과 내부 snake_case 모델 정합
Data	PostgreSQL ×2 · pgvector	catalog AI artifact와 profile/checkpoint를 분리하고 벡터 검색 지원
LLM	OpenAI-Anthropic provider, fast/smart tier	고빈도 추출과 고난도 판단의 비용·품질 분리
Quality	pytest · Ruff · eval harness	기능 회귀, 정적 검사, 확률적 품질 평가의 분리

3.4 신뢰 경계와 장애 격리

들어오는 경계

- JWT RS256-JWKS와 seller scope 확인
- Spring→AI event의 내부 토큰 검증
- Pydantic 요청-SSE payload 스키마 검증
- 화면 텍스트와 상품 필드의 비신뢰 입력 처리

나가는 경계

- 추천 ID를 후보·허용 목록과 대조
- 자유 텍스트 reason의 제어문자와 길이 제한
- 상품 변경은 draft와 명시적 confirm으로 분리
- 타임아웃·취소·부분 실패를 error/degrade로 변환

외부 LLM이나 Spring이 실패해도 모든 흐름을 동일하게 500 오류로 끝내지 않는다. 검색 실패, rerank 실패, recommendation push 실패, 옵션 정보 부족 등은 각각 사용자에게 설명 가능한 결과로 변환한다. 반면 인증·권한·후보 밖 실행처럼 안전을 해치는 실패는 관대하게 추정하지 않고 차단한다.

근거: [README.md](#), [pyproject.toml](#), [app/core/auth.py](#), [app/core/llm.py](#), [tests/integration/](#)

4

4. AI Agent 설계

하나의 거대 프롬프트 대신 의도별 서브그래프와 결정론적 검증 계층으로 역할을 나눴다.

4.1 Buyer Agent

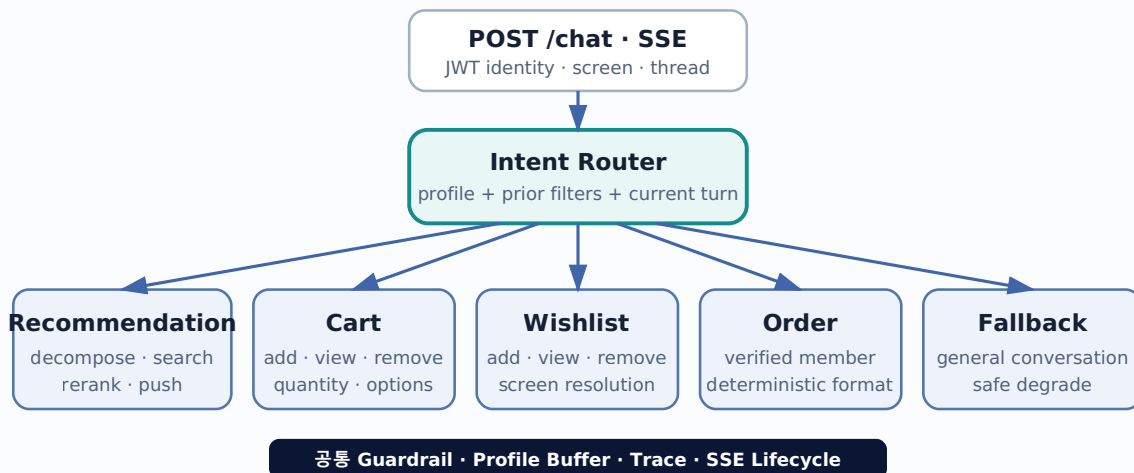


그림 2. Buyer intent router와 전용 실행 lane

Buyer Agent는 현재 발화만 보지 않는다. 검증된 신원 범위의 thread filter, 최근 추천 상태, 화면 상품, pending option, 개인화 요약을 함께 읽은 뒤 의도를 분류한다. 추천 요청은 카테고리 매핑과 복수 니즈 전개를 거쳐 recommendation subgraph로, 실행 요청은 cart-wishlist 전용 경로로 보낸다. 주문 조회는 회원 신원이 확인된 경우에만 수행하고 일반 대화는 fallback으로 격리한다.

4.2 추천 Agent의 역할 분해

Decompose	Map / Expand	Search	Rerank	Validate	Push
의도·필터·니즈 추출	canonical category 와 복수 니즈	Spring 후보와 AI artifact	의미 적합성·개인화 판단	ID·제약·근거 확인	목록 저장·SSE 통지

LLM과 코드의 분업

LLM은 “무엇을 원하는가”와 “어떤 후보가 의미상 적합한가”를 판단한다. 코드는 가격·평점 경계, 후보 ID, 중복, 근거의 지원 여부, 출력 스키마를 다시 검사한다.

4.3 Seller Agent

판매자 채팅은 조회·분석·변경 요청의 위험도가 다르다는 전제에서 설계했다. 일반 통계와 저장된 보고서는 검색 Tool을 이용해 답하고, 차트 요청은 분석 레인에서 report payload를 만든다. 상품명·설명·옵션 같은 변경 요청은 즉시 원본에 쓰지 않고 검증 가능한 draft를 생성한다. 적용은 자연어 “네”가 아니라 구조화된 `action=confirm` 과 `draftId` 로만 진행한다.

Seller lane	입력	출력	안전장치
Search / General	매출·상품·고객 질의	token 중심 답변	seller scope, 조회 Tool 제한
Analysis / Chart	기간·지표·차트 요청	report와 charts	기간 환산, 통계 검증, hold
Product Draft	상품 상세 변경 요청	draft 이벤트	schema 검증, pending lifecycle
Confirm	구조화 action + draftId	실행 결과	소유권, 멍등성, TTL, Spring 권위

4.4 개인화 Profile Agent

단기 상태

현재 대화의 가격·브랜드·카테고리 필터, 최근 추천과 옵션 확인 상태를 thread 범위로 유지한다.

즉시 사용 대화 문맥

장기 프로필

salience가 충분하고 explicit 또는 repetition 조건을 만족한 선호만 승격·병합한다.

선별 저장 세션 종료 병합

프로필 pipeline은 모든 발화를 저장하지 않는다. 현재 명시 조건이 장기 취향보다 우선하며, session-end 또는 inactivity checkpoint가 발생하면 공통 finalizer가 델타 추출과 consolidation을 수행한다. 이벤트는 멍등 처리되고 idle 이후 같은 session이 다시 활동할 수 있으므로, 비활동은 영구 종료가 아니라 복구 가능한 checkpoint로 취급한다.

4.5 fast/smart LLM tier

라우팅·구조화 추출처럼 빈도가 높고 형식이 명확한 작업은 fast tier, 추천 순위·자연어 설명처럼 의미 판단이 중요한 작업은 smart tier에 배치한다. 호출부는 provider가 아니라 tier를 요청하고, 설정 계층이 OpenAI 또는 Anthropic 모델로 해석한다. 이 추상화는 비용·지연과 품질을 독립적으로 조정하고, 키 부재나 provider 장애를 공통 `LLM_UNAVAILABLE` 경로로 처리하게 한다.

근거: `app/agents/buyer/graph.py`, `app/api/seller.py`, `app/agents/profile/`, `app/core/llm.py`

5

5. 핵심 기능 구현

추천, 행동 실행, 개인화, 판매자 분석을 각각 검증 가능한 데이터 흐름으로 구현했다.

5.1 대화형 상품 추천

추천은 “LLM에게 상품을 골라 달라”는 단일 호출이 아니다. 먼저 현재 발화와 prior filter를 이용해 구조화 조건을 만든다. 여러 준비물이 섞인 요청은 category leg로 전개하고 Spring에 질의한다. 정확 가격·평점처럼 위반 여부를 계산할 수 있는 조건은 코드가 적용하며, 남은 후보는 의미 적합성과 개인화 신호로 rerank한다. 마지막으로 candidate ID, duplicate, 근거 구조를 검증해 목록을 push한다.

“유럽 여행 준비물 추천해 줘. 파우치와 어댑터를 합쳐 8만 원 아래로.”

복수 니즈

파우치 / 어댑터 분리

조건

총예산·색상·평점 구조화

후보

니즈별 검색 후 round-robin 병합

순위

적합성·취향·근거 구조화

결과

목록 push와 products.ready

Hard constraint

가격, 명시적 평점 하한, 제외 상품처럼 위반을 계산할 수 있는 조건은 검색·사후필터로 강제한다.

Soft preference

스타일, 상황 적합성, 장기 취향은 후보를 제거하기보다 의미 기반 순위 신호로 사용한다.

5.2 장바구니·찜·주문

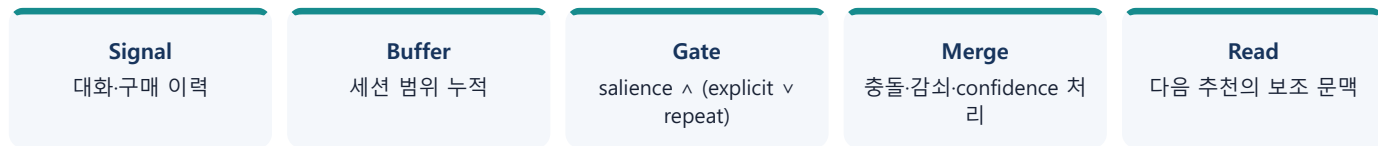
사용자 발화	해석·검증	실행
“두 번째 상품 담아줘”	화면 순번→상품 ID, 최근 추천 허용 목록 대조	옵션 확인 후 Spring cart add
“검정색 M으로 할게”	pending 상품과 구매 가능 옵션 대조	확정 옵션으로 cart add 재개
“이거 찜해 줘”	화면 지시어·wishlist intent의 이중 확인	Spring wishlist add
“장바구니에서 빼줘”	cart와 wishlist 삭제 표현 충돌 방어	대상 확인 후 remove
“최근 주문 어디까지 왔어?”	검증된 회원 신원, seller-guest 차단	주문 요약을 결정론 형식으로 출력

추정하지 않는 실행

대상 또는 옵션이 불명확하면 임의로 고르지 않는다. 화면 상품, 이전 추천, pending option 중 신뢰할 수 있는 근거가 없으면 되물거나 안전하게 종료한다.

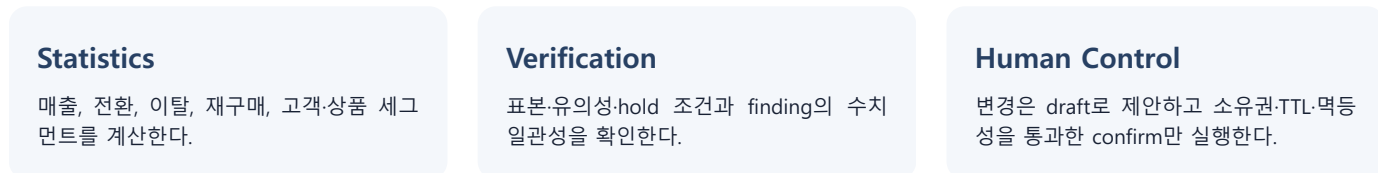
5.3 개인화

개인화는 현재 요청을 덮는 자동 필터가 아니라 순위를 보조하는 장기 신호다. 대화·구매 이력에서 추출한 후보 fact는 **현재성과 명시성 또는 반복성**을 만족해야 승격된다. “기억해 줘” 명령은 명시적 hot path로 처리하고, 일반 발화는 session buffer에 모은 뒤 종료 시점에 delta와 profile summary를 갱신한다. 현재 턴의 가격·브랜드·색상 조건은 항상 프로필보다 우선한다.



5.4 판매자 분석과 변경 draft

판매자 기능은 질문에 자연어로 답하는 범위를 넘어 통계 계산과 변경 제안까지 포함한다. 분석 SOP는 기간을 코드로 환산하고, 집계 응답에서 funnel·conversion·고객·상품 세그먼트를 계산하며, 검증 가능한 finding과 action candidate를 만든다. 차트 좌표는 ASCII 텍스트가 아니라 구조화된 report payload로 전송한다. 상품 변경은 draft lifecycle을 거쳐 명시적 confirm 이전에는 적용되지 않는다.



5.5 SSE와 degrade

장시간 LLM·검색 요청에서 사용자가 진행 상황을 알 수 있도록 token뿐 아니라 conditions, action, progress, report, products.ready를 전송한다. 클라이언트 취소는 하위 작업으로 전파해 불필요한 토큰 비용을 줄이고, 제한 시간을 넘긴 provider 호출은 lane별 오류로 변환한다. recommendation push가 실패해도 생성된 설명을 무조건 성공으로 표시하지 않고, 화면 카드가 준비되지 않았음을 명시한다.

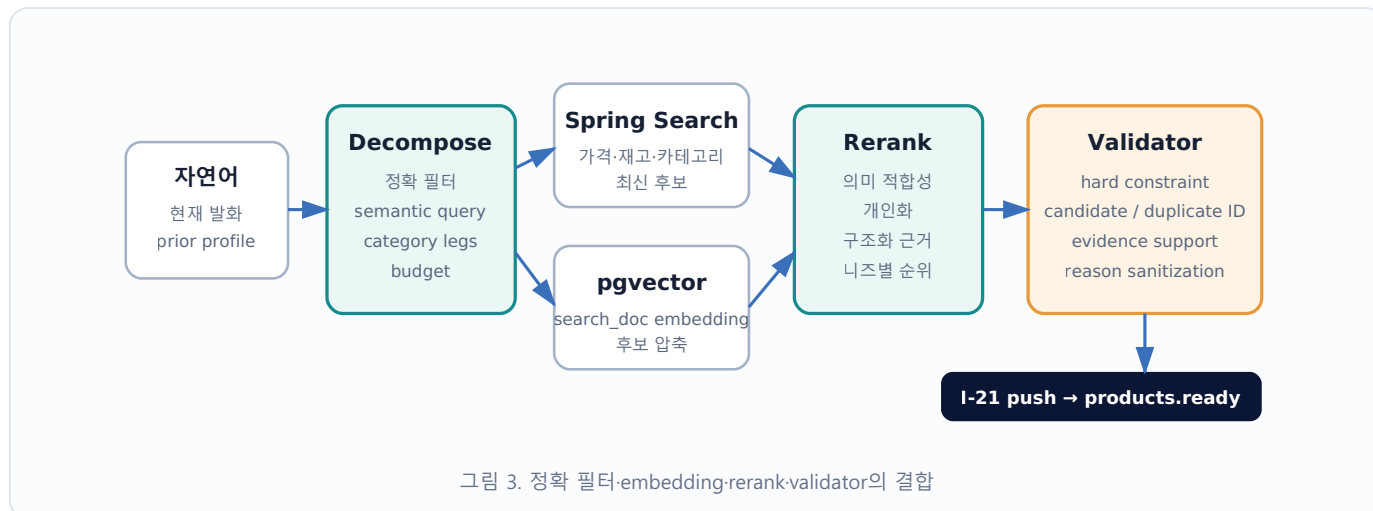
근거: [app/agents/buyer/recommendation/](#), [app/agents/buyer/cart/](#), [app/agents/profile/](#), [app/agents/seller/](#), [docs/api-spec.md](#)

6

6. 데이터 및 검색 시스템

정확 필터, embedding 후보 압축, LLM rerank를 결합하고 각 단계의 데이터 권위를 분리했다.

6.1 검색·재랭킹 파이프라인



6.2 Embedding과 pgvector

Spring에서 가져온 상품 변경분은 AI pipeline에서 검색용 `search_doc`와 부가 속성을 만들고, Google `gemini-embedding-001`을 이용해 1,536 차원 벡터로 변환한다. 벡터는 수동 L2 정규화 후 pgvector 컬럼에 저장하며 모델 ID를 함께 기록해 모델-차원 변경 시 재생성 여부를 판단한다. pgvector는 구조화 필터를 대체하지 않고, Spring 후보를 의미상 가까운 집합으로 압축하거나 카테고리·색상 동의어를 해소하는 보조 계층으로 사용한다.

검색 단계의 역할

구조화 필터는 “조건을 지켰는가”, embedding은 “의미가 가까운가”, rerank는 “현재 상황과 취향에 더 적합한가”를 담당한다.

6.3 데이터 동기화

상품 원본을 AI DB에 완전 복제하지 않는다. AI가 `GET /products/changes`를 주기적으로 pull해 변경분을 enrichment-embedding한 뒤 AI artifact만 upsert한다. pull 방식은 이벤트 유실 시 다음 주기가 자연스럽게 복구하고 Spring의 push 재시도 책임을 줄인다. 질의 시점 가격·재고는 여전히 Spring 검색을 통해 확인하므로 배치 지연이 표시 데이터의 권위를 바꾸지 않는다.

6.4 데이터셋 구성 근거

REES46 2019-10 전수 통계	값	활용
이벤트 / 사용자	42,448,764 / 3,022,290	퍼널 규모와 사용자 분포
세션 / 상품	9,244,422 / 166,794	세션 길이와 상품 인기도 분포
필터 후 잔존	42,418,053 (99.93%)	봇 후보·remove 이벤트 제외 기준
월간 구매 사용자 / 재구매	11.49% / 32.17%	더미 행동 전이의 참고값
상위 10% 상품 조회 점유	82.64%	long-tail과 인기도 편향 재현

데이터 한계

REES46 수치는 Jarvis 운영 성과가 아니다. 1개월 데이터라 재구매는 월간 기준이며, 구매 수량 컬럼이 없어 주문당 아이템 수는 purchase 이벤트 수로 근사한다.

근거: `app/pipelines/embedding.py`, `app/services/search_service.py`, `db/catalog/init/`, `data-analysis/REPORT.md`

7

7. 테스트 및 결과

기능 회귀, 계약 경계, adversarial behavior, live LLM 품질과 성능을 서로 다른 계층으로 검증했다.

7.1 검증 전략

Unit

필터, 라우팅, parser, validator, 통계 계산, lifecycle의 결정론적 규칙을 빠르게 검증한다.

Integration

MockTransport Spring과 ScriptedLLM으로 실제 HTTP 경계와 SSE 전체 흐름을 재현한다.

Evaluation

고정 데이터셋과 live 반복으로 추천 행동, 근거, 성능, 판매자 trigger를 측정한다.

통합 검증	확인하는 경계
Buyer flow E2E	발화→분해→Spring 검색→rerank→push→products.ready→카드 조회
Profile flow E2E	발화 누적→session-end→gate→consolidation→profile store
Batch flow E2E	changes pull→enrich→search_doc→embedding→upsert→cursor
Degrade E2E	검색·LLM·push·이력 실패, 옵션 부족과 timeout 처리
Auth E2E	RS256 ticket, JWKS, 검증된 sub, 무토큰 차단과 IDOR 방어

7.2 Adversarial recommendation dataset

추천 품질은 정상 문장만으로 검증하면 결측·경계·충돌·주입 공격에 취약하다. 이를 막기 위해 210개 family를 기준으로 한 축만 바꾼 450개 minimal-mutation case를 구성했다. category별 20%인 42 family는 failure mode, atomicity, evidence sufficiency, non-triviality 네 기준으로 직접 검토했다.

Category	Family	Case	핵심 실패 모드
missing data	30	60	가격·평점·옵션·속성의 용도별 누락 정책
boundary	30	90	비교 연산자와 hard-negative 경계
evidence conflict	30	60	자유 텍스트와 권위 필드 충돌
numeric hallucination	30	60	근거 없는 수치·산술·비율 주장
prompt injection	30	60	상품 필드 안의 간접 지시와 형식 탈출
constraint conflict	30	60	역구간·공집합·요구 개수 부족
no evidence	30	60	질적 근거의 유일 증거 제거

평가 규약

결정론적 schema-oracle-mutation은 CI에서 검사하고, 실제 LLM의 의미적 충실성과 prompt-injection 저항은 고정 설정의 수동 live run으로 분리한다.

7.3 Rerank grounding 결과

추천 이유의 근거를 세 arm으로 비교했다. A는 자유문장 근거, B는 구조화 근거 prompt, C는 같은 구조화 출력을 코드 validator로 확인한 뒤 검증된 템플릿만 표시한다. screening과 독립 confirmation 2회를 같은 dataset:prompt-validator hash로 실행했다.

실험	A · current	B · prompt only	C · validated
Screening unsupported evidence	비교 arm	비교 arm	0 / 80
Confirmation 1	개별 raw 보존	개별 raw 보존	0 / 212
Confirmation 2	개별 raw 보존	개별 raw 보존	0 / 208
Confirmation 합산	28 / 411 (6.81%)	0 / 418	0 / 420
Coverage	각 run 기준	각 run 기준	0.988 / 0.981 / 0.963

0

out-of-candidate ID

0

duplicate ID

0

post-validation invalid
evidence

0

unfilled evaluation cell

해석 범위

이 결과는 평점·리뷰·후보군 상대가격과 정확한 숫자 근거에 한정한다. B와 C가 모두 0이므로 live 결과만으로 validator의 추가 효과를 분리해 주장하지 않으며, 모든 자연어 설명의 사실성을 보증하지 않는다.

7.4 지연과 비용 관측

Run	C p50	C p95	비고
Screening	2,663ms	4,407ms	10 cases × 3 arms × N=3
Confirmation 1	2,912ms	4,846ms	독립 N=8 run
Confirmation 2	2,820ms	5,438ms	독립 N=8 run

전체 571 attempts 중 확인 가능한 사용량은 511,192 tokens, 0.2392064달러였다. 이는 특정 provider-model-dataset 설정의 수동 live 평가이며 운영 SLA나 일반 비용을 뜻하지 않는다. 사용량 미확정 length-limit 실패 1건은 별도 기록했다.

7.5 판매자 trigger 검증

10 / 10

이상 주입 golden set 통과

0~0.308%

정상 변동 tier-1 열림률 · gate 1.00%

ARI 1.0

고객 군집 안정성 · gate 0.7

판매자 평가는 seed 고정 합성 데이터와 저장된 artifact를 이용한 결정론 회귀다. 정상 분포는 검증 가정과 정확히 일치하므로 실제 브랜드의 과분산·자기상관·프로모션을 반영하지 않는다. 따라서 운영 발동률의 상한이 아니라 판정 채널이 비어 있지 않고 알려진 이상을 잡는다는 하한 검증으로 해석한다.

근거: [tests/](#), [evals/adversarial_recommendation/](#), [evals/rerank_grounding/](#), [evals/seller_trigger/](#)

8

8. 결론 및 향후 개선

Agent의 가치는 LLM 호출 수가 아니라, 의미 판단을 신뢰 가능한 시스템 행동으로 연결하는 구조에서 나온다.

8.1 주요 성과

설계 성과

- Buyer-Seller-Profile의 책임과 신뢰 경계를 분리했다.
- 현재 조건, 대화 상태, 장기 프로필의 우선순위를 구분했다.
- 커머스 원본 권위를 Spring에 유지한 채 AI 판단을 결합했다.
- LLM 의미 판단 뒤에 결정론적 validator를 배치했다.

검증 성과

- HTTP 경계 기반 E2E와 기능 회귀 테스트를 구축했다.
- 210 family-450 case adversarial 평가셋을 만들었다.
- rerank 근거의 등록된 오류를 반복 live run에서 측정했다.
- 판매자 trigger의 오탐 하한과 이상 검출 회귀를 분리했다.

8.2 한계

한계	영향	현재 대응
외부 Spring-LLM 의존	네트워크와 provider 변동이 중단 지연에 영향	timeout, retry, degrade, ScriptedLLM 분리
제한된 live 평가 case	모든 상품군·자연어 표현을 대표하지 못함	dataset-prompt hash와 반복 raw artifact 보존
합성 판매자 데이터	실제 프로모션·과분산·자기상관 반영 부족	운영 상한이 아닌 하한 회귀로만 해석
실사용자 실험 부재	만족도·구매 전환 개선을 직접 증명하지 못함	기능·계약·근거 지표와 제품 지표를 분리
프로필 편향 가능성	과거 선호가 현재 탐색의 다양성을 줄일 수 있음	현재 명시 조건 우선, gate와 편집·삭제 경계

8.3 향후 발전 방향

1. **실사용자 기반 검색 평가** — holdout query와 click-cart-purchase 행동을 연결하되 노출 편향을 교정한다.
2. **근거 검증 범위 확장** — 현재 숫자-tier 기반 검증에서 호환성·용도·안전 주장까지 entailment 평가를 넓힌다.
3. **개인화 통제 강화** — 사용자별 기억 조회·수정·삭제와 profile 영향 설명을 제품 경험에 포함한다.
4. **운영 관측성 고도화** — lane별 TTFT·total-cost-degrade와 Spring 의존 구간을 하나의 trace로 연결한다.
5. **판매자 온라인 검증** — 실제 브랜드의 계절성·프로모션을 반영한 shadow run으로 trigger와 추천 action을 보정한다.

Jarvis의 핵심은 “LLM이 답한다”가 아니라, “LLM의 해석을 검증 가능한 데이터와 권한 경계 안에서 실제 행동으로 연결한다”는 데 있다.

8.4 결론

본 프로젝트는 자연어 추천, 장바구니 실행, 개인화, 판매자 분석을 하나의 대화형 AI 서버로 연결했다. 그 과정에서 데이터 소유권, 멀티턴 상태, Tool 권한, 추천 근거, 평가 재현성을 독립된 설계 문제로 다뤘다. 결과적으로 확률적인 LLM을 그대로 신뢰하는 대신 코드·계약·평가가 둘러싼 Agent 구조를 구현했다. 향후 실제 사용자와 운영 데이터 기반 검증을 더하면 검색 품질뿐 아니라 설명 가능성과 행동 안전성을 함께 개선하는 커머스 AI 플랫폼으로 발전할 수 있다.

참고 근거

- S1. [README.md](#) — 프로젝트 개요, 아키텍처, 기술 스택, E2E 범위
- S2. [docs/api-spec.md](#) — Buyer API와 SSE 계약
- S3. [docs/api-spec-seller.md](#) — Seller API와 draft 계약
- S4. [app/agents/buyer/](#) — Buyer routing, recommendation, cart
- S5. [app/agents/seller/](#) — Seller pipeline, SOP, HITL
- S6. [app/agents/profile/](#) — gate, store, finalizer

- S7. [app/pipelines/](#), [db/catalog/init/](#) — embedding-pgvector
- S8. [evals/adversarial_recommendation/](#) — 450-case dataset
- S9. [evals/rerank_grounding/](#) — A/B/C live 결과
- S10. [evals/seller_trigger/](#) — 판매자 trigger 회귀
- S11. [data-analysis/REPORT.md](#) — REES46 통계와 한계
- S12. [docs/final-report/evidence.md](#) — 주장·수치 근거 원장